



STUDENT

0016-FEF

TENTAMEN

250118 DV2606 2110

Salstentamen (DI)

Kurskod	--
Bedömningsform	--
Starttid	18.01.2025 11:00
Sluttid	18.01.2025 16:00
Bedömningsfrist	--
PDF skapad	05.11.2025 17:12
Skapad av	Anders Göransson

- 1.1 You are parallelizing a sequential program. Assume that you manage to parallelize 95% of the sequential program, i.e., 5% is still executed sequentially. What is the highest speedup that you can archive (given an infinite number of processors)?

Select one alternative:

☐ None of the other alternatives is correct

☐ Infinite

☐ 95

☐ 5

☒ 20



Rätt. 1 av 1 poäng.

- 1.2 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

☒ Put more barrier synchronizations in the code.



☐ Increase the number of processors.

☐ Decrease the input data set size.

☐ All alternatives decrease the efficiency.



☐ Increase the clock frequency of the processors.

Fel. 0 av 1 poäng.

1.3 *SPMD* is a common programming model for writing parallel programs. *SPMD* stands for:

- ☐ Single Processor Multiple threads
- ☐ None of the alternatives is correct.
- ☐ Shared-memory Programming for Multiple Data
- ☐ Shared-memory Program Many Distributions
- ☒ Single Program Multiple Data



Rätt. 1 av 1 poäng.

1.4 Simultaneous multithreading is ...

- ☒ a technique to exploit parallelism in the instruction execution pipeline
- ☐ a technique to implement multi-level caches
- ☐ none of the alternatives is correct
- ☐ a software approach to parallelize an application
- ☐ a technique to improve branch prediction



Rätt. 1 av 1 poäng.

1.5 Why are snoopy-based cache coherence protocols not used in large-scale shared-memory multiprocessors?

- ☒ They are not scalable to a large number of nodes. 
- ☐ They can only be implemented using a write-update policy.
- ☐ Modern processors are too fast for snoopy-based protocols.
- ☐ None of the alternatives is correct.
- ☐ They can not guarantee coherence for more than ten caches.

Rätt. 1 av 1 poäng.

1.6 MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ The request is denied since the block is in state Modified.
- ☒ The cache replies with the requested block, and the block stays in state Modified. 
- ☐ None of the alternatives is correct. 
- ☐ The cache replies with the requested block, and the block is marked as Invalid.
- ☐ The processor is always stalled until the bus request is handled.

Fel. 0 av 1 poäng.

1.7 A fundamental difference between a directory-based cache coherence protocol and a snoopy-based cache coherence protocol is ...

- ☐ All of the alternatives are correct.
- ☒ A directory-based protocol can only be implemented using the write-invalidate policy 
- ☐ A directory-based protocol keeps exact information in which caches copies of each memory block reside. 
- ☐ A directory-based protocol relies on a shared broadcast medium.
- ☐ A directory-based protocol can only be implemented using the write-update policy.

Fel. 0 av 1 poäng.

1.8 What is the (asymptotic) parallel time complexity of *matrix-matrix multiplication* using n processors, assuming $n \times n$ matrices?

- ☐ $O(n \log n)$
- ☐ $O(\log n)$
- ☒ $O(n^2)$ 
- ☐ None of the alternatives is correct.
- ☐ $O(n)$

Rätt. 1 av 1 poäng.

1.9 A PRAM (Parallel Random Access Machine) can be useful for ...

- ☐ All of the alternatives are correct.
- ☐ Running applications with very large input data sets.
- ☐ Very easy implementation of parallel hardware.
- ☐ Running message-passing applications with GPU support.
- ☒ Designing and evaluating parallel algorithms.



Rätt. 1 av 1 poäng.

2.1 Describe *Flynn's taxonomy* of different computer architectures.

Flynns taxonomy divides processors into four categories, based on how many instructions they execute and how they read data

SIMD - single instruction multiple data:

Most GPU cores works using this method

SISD - single instruction single data:

Traditional uniprocessors works this way, uncommon in computers today

MIMD - multiple instructions multiple data:

Modern CPU;s usually utilizes this with simultaneous multithreading

MISD - multiple instructions single data:

Not really used but could hypothetically be used for data pipelines

Ord: 78

Besvarad.

- 2.2** Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

UMA:

It takes all processors approximately the same time to access the shared memory

NUMA:

It takes different amount of time for the processors to access the shared memory.

This difference is usually due to varying distance between the processors and the memory

NUMA could be advantageous if you are working with a system that is expected to scale since it easier to add processors when you don't have to follow the UMA constraint

UMA would be better when working with data parallelization approaches that synchronize, since having to synchronize would force the processors with faster memory access to idle until the ones that are slower are done. Effectively making the program as slow as the processor with the slowest memory access.

Ord: 122

Besvarad.

2.3 When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

Task parallelism:

Task parallelism divides the program into "tasks" and divide them between processing units.

Advantages:

- Can be intuitive to work with
- The best approach when working with NUMA structures since we can give the processors with the slowest memory access tasks that require less memory access

Disadvantages:

- Can run into scenarios where one task is dependent on another forcing processing units to idle until the required task is done
- Harder to properly balance the load between processing units since different tasks require different load

Suitable task:

An application with user interface and backend computations would be a suitable task for task parallelism since there are clear separation between tasks such as backend computations, graphics and user control. Each of these tasks could of course be broken down further, into smaller tasks for parallelization.

Data parallelism:

Data parallelism works by dividing the data between processing units, giving each a set of data to compute, with the same instructions in parallel to the others.

Advantages:

- Easy to do load balancing since the programmer determines how much data each processing unit receives.

Disadvantages:

- Some tasks simply isn't suited for data parallelism due to how data needs to interact with other data, e.g normalizing an array

Suitable task:

Matrix multiplication works well with data parallelization since there are few calculations that need to be performed on a (usually) large set of data. The data is also easily divided between processes and each calculation can be done by only accessing a specific part of the matrix.

Ord: 252

Besvarad.

- 2.4** When we parallelize a sequential application, e.g., by using threads or message-passing, so it can execute in parallel on N processors, we often find that the parallel program is not N times faster. Describe two important reasons for that.

Compute overhead:

Initializing threads or dividing the application into parts for each thread requires compute time by itself, creating more computations that doesn't allow for parallelization.

Sequential parts/redundant calculations:

Sometimes there are data that every thread needs. If this data needs to be calculated we are left with two options:

Letting all threads calculate the information by themselves

Computing the data sequentially and then passing it as arguments to the threads.

In both scenarios we essentially create a sequential part that can't be parallelized and thus not utilizing the parallelization.

Ord: 90

Besvarad.

- 2.5** Sometimes we find that a system or application has a *superlinear speedup*. Clearly describe two situations when and why it may occur.

Cache efficiency:

If we have a scenario where we parallelize a task that needs to read the same data each time at specific times during its execution, it is more likely that we need to read the data to the cache only once and each thread reads from the cache before the data is replaced with other data. If we would have had a sequential loop, it is likely that we would have needed to read in this data each iteration since the cache would have been replaced by the time the sequential program got to that point again.

Less computations needed:

Sometimes parallelization creates scenarios where there are less computations needed to be done in total. E.g. when searching a binary tree there are scenarios where parallelization allows a thread to "stumble" upon the value we are looking for after less total steps than if we would have had to search the tree with a sequential algorithm.

Ord: 158

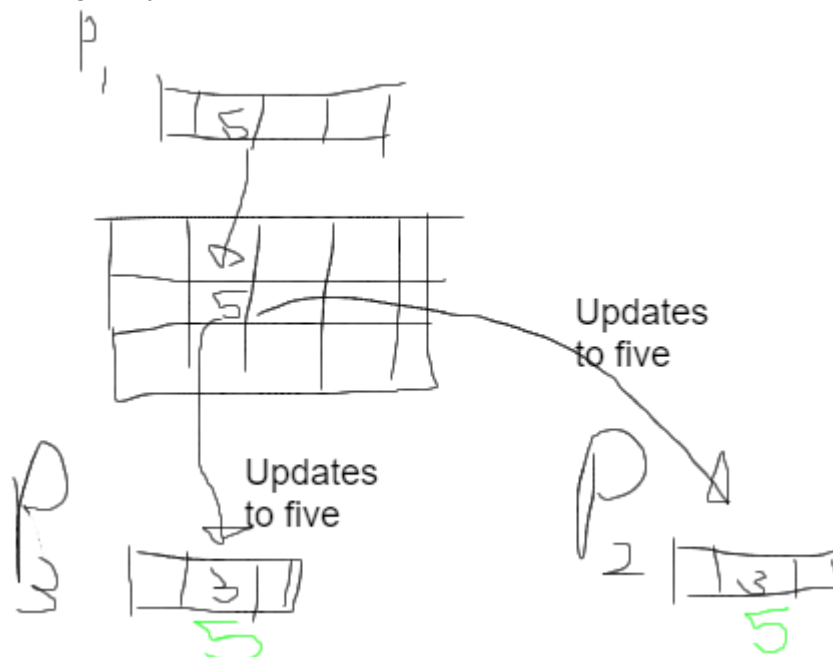
Besvarad.

- 2.6 Cache coherence** is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)
 Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)
 Describe some advantage and disadvantage with each of the approaches. (1p)

Let's say we have N number of processors, all reading the value 3 to their cache from the shared memory. If one of these processors changes the value to 5 and then saves it to the shared memory, the values all other processors have saved in their caches is no longer up to date. All calculations where these processors would have used the cached value (3) would then have been wrong. That is why cache coherence is important.

Write update:

This protocol works by updating all caches to the correct value after the value has been changed by one processor



Advantage:

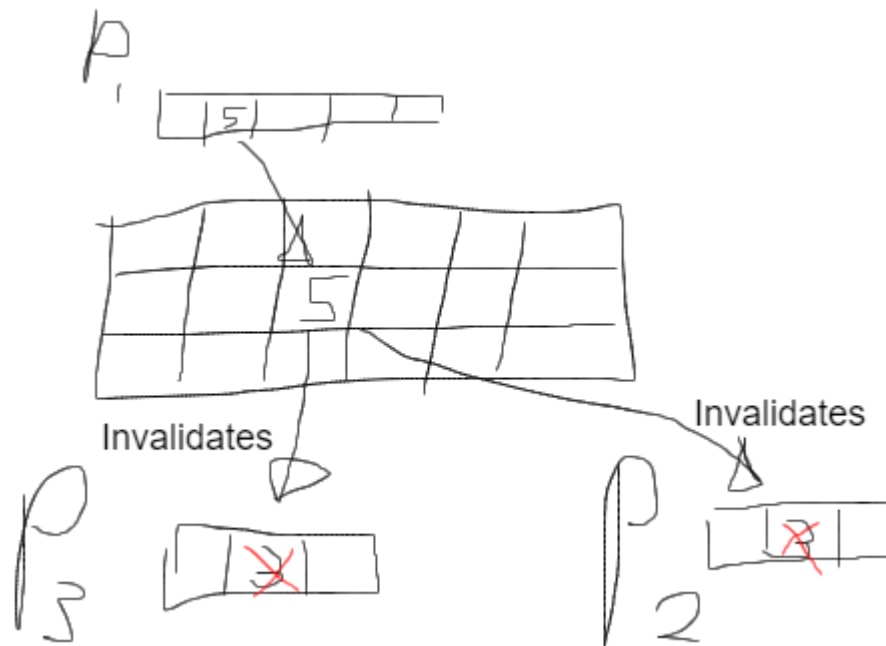
In programs that reads much more than they update values, this solution is effective since it avoids cache misses due to invalid values, which can be quite computationally expensive

Disadvantage:

If other processes doesn't need to read values between frequent updates by one process, this can be quite inefficient since we update all caches when we don't really need to

Write invalidate:

Write invalidate sends a signal that invalidates the value in all caches. It is then the processors own responsibility to update the value when they try to read it from the cache later on

**Advantage:**

Does not require the processor that made the change to update all caches, making changes to values less computationally expensive

Disadvantage:

It is more expensive to update the value by invalidate once one of the other processes actually need it, making it inefficient if multiple processors are expected to read the value between updates.

Ord: 252

Besvarad.

3.1 Thread coarsening

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ Thread coarsening decreases the completion time of a thread
- ☒ Thread coarsening can reduce the number redundant calculations in a CUDA program 
- ☐ Thread coarsening increases the parallelism in a CUDA program
- ☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program 
- ☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU

Rätt. 2 av 2 poäng.

3.2 Reduction sum

Consider the two functions SumReduction1 and SumReduction2 below:

```
__global__ void SumReduction1(float* input) {
    int i = 2*threadIdx.x
    for (int stride = 1; stride <= blockDim.x; stride *= 2) {
        if (threadIdx.x % stride == 0) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}

__global__ void SumReduction2(float* input) {
    int i = threadIdx.x
    for (int stride = blockDim.x; stride >= 1; stride /= 2) {
        if (threadIdx.x < stride) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}
```

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ SumReduction1 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction2. 

☒ SumReduction2 has less control divergence than SumReduction1 

☐ SumReduction2 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction1. 

☐ SumReduction1 has less control divergence than SumReduction2.

☐ SumReduction1 uses a gather approach and SumReduction2 uses a scatter approach.

Delvis rätt. 0 av 2 poäng.

3.3

Consider the following small program.

```
for (int m = 0; m < M; m++) {
    input[m].value += 5;
    for (int n = 0; n < N; n++) {
        output[n].value += input[m].value / distance(input[m].position, output[n].position);
    }
}
```

If we transform this program into a parallel CUDA program, we may run into some serious performance problems in the CUDA program.

What kind of performance problems may the CUDA program suffer from? How could we modify the small program above so that it is still functionally equivalent and so that the corresponding CUDA program does not suffer from any serious performance problems?

If we would try to parallelize this with respect to the outer loop we would have to deal with the problem that all threads would try to modify all elements in the array output. This would either create race conditions or would require atomic operations, something that is very inefficient.

The solution would be to move the line "input[m].value += 5" and parallelize it by itself. We would then have to swap the inner and outer loop of the function and parallelize with respect to the "n-loop" giving each thread it's own "n value" and thus it's own value in the output array to modify. This would then be a gather operation. it would look something like this

```
__global__ void AddFive(){
    int id = threadIdx.x;
    for(i = id; i < M; i += numThreads){
        input[m].value += 5;
    }
}

__global__ void MainCalc(){
    int id = threadIdx.x;
    for(n = id; n < N; n += numThreads){
        for(int m = 0; m < M; m++){
            output[n].value += input[m].value / distance(input[m].position, output[n].position);
        }
    }
}

AddFive<<numThreads, numBlocks>>>();

MainCalc<<numBlocks, numThreads>>>();
```

Ord: 175

Besvarad.

3.4

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ Context switching on a GPU is normally faster than context switching on a CPU. 

☐ The processor cores on a CPU are normally organized in streaming multiprocessors.

☐ A CPU normally has more processor cores than a GPU.

☐ The processor cores in a CPU normally execute in SIMD mode.

☒ A CPU normally has larger caches than a GPU. 

Rätt. 2 av 2 poäng.

3.5

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☐ All the threads in a thread block see the same registers

☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 

☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.

☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 

☐ The CPU can access variables in GPU shared memory

Rätt. 2 av 2 poäng.

3.6 Cuda Streams

What are Cuda Streams? How can they be used to improve performance? Give a short example.

Cuda streams is a way to move data between memory and GPU while also performing calculations on the GPU. This avoids a scenario where the GPU is idle, waiting for the memory bus or vice versa.

One example where one could use this would be to calculate the moving average of an array. The GPU could calculate the average for a specific point within a window, while the bus delivers the previous result to an output array and fetch the necessary data for the next calculation the GPU will perform.

Ord: 90

Besvarad.

3.7 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☐ Threads in the same Warp can execute on different Streaming Multiprocessors.

☒ Threads in the same Warp cannot execute different instructions at the same time. 

☐ Threads from different Thread Blocks can execute in the same Warp.

☐ A Thread Block cannot be larger than a Warp.

☒ A Thread Block can contain many Warps. 

Rätt. 2 av 2 poäng.

3.8 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privatization is a technique where each thread has its own copy of an output to minimize the amount of atomic operations needed. For example, if we have a matrix that the threads should summarize to one value. Each thread is given some part of the matrix. Without privatization each thread would have to perform an atomic add operation to the output value for each value in their part of the matrix. However with privatization each thread summarizes their part of the matrix in a local variable and then performs only one atomic add operation to the output value, ensuring that we have only as many atomic add operations as we have threads instead of having as many atomic add operations as we have elements in the matrix making the entire process much more efficient since atomic operations are really inefficient for performance.

Ord: 142

Besvarad.

3.9 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.

Tiling is a technique where we load only a part of the data needed for the entire operation. We then do parts of the calculations needed with this data and then load the next part and repeats the process. This ensures that we can fit all the data on the shared memory and do not need to load from the global memory which is much more inefficient. However we need to synchronize the threads before loading the next part since all threads need to be done with the currently loaded part before we load the next one.

Ord: 97

Besvarad.

3.10

Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM. Why do we normally want to have high occupancy? Which hardware related parameters affect occupancy? Discuss and give examples of how we as programmers, by taking smart design decisions, can provide good trade-offs between the parameters that affect occupancy.

High occupancy ensures that we fully utilize the processing resources we have at our disposal, making the application much more efficient. Some parameters that affect occupancy is size of shared memory, maximum number of threads on an SM, maximum number of blocks, size of registers and size of cache.

Some examples of trade off:

Having high occupancy on a SM of course utilizes the amount of threads and computation we can perform but with a limited cache, this can result in us having to write over the cache more often. Essentially making our program memory bound instead.

Another example would be the shared memory. Let say each thread requires X amount of bytes of data to perform it's calculations. We then want to have as many threads as possible while still fulfilling the requirement: $X * (\text{number of threads}) < (\text{available bytes on shared memory})$. This might not utilize the computing resources fully but will ensure that we don't have to read from global memory as often.

Ord: 165

Besvarad.